Evaluation Metrics

1. The Intuition: Beyond Accuracy

"My model is 99% accurate!" ... "Great, but it just predicts 'Healthy' for everyone, including the 1% who have cancer."

Accuracy can be misleading, especially with imbalanced data. We need better tools:

- **Precision:** "Of all the chapters I studied, how many were actually on the exam?" (Did I waste time on irrelevant stuff?)
- **Recall (Sensitivity):** "Of all the topics on the exam, how many did I actually study?" (Did I miss important material?)
- **F1 Score:** "The harmonic mean—finding the balance between efficient studying (Precision) and covering everything (Recall)."

2. Interactive Confusion Matrix

Adjust the values to see how metrics change.

	Predicted Class 1 (Positive)	**Predicted Class 0 (Negative)**
Actual Class 1 (Positive)	**True Positive (TP)** Class 1 correct as Class 1 50	**False Negative (FN)** Class 1 wrong as Class 0 10

Actual Class 0 (Negative)	**False Positive (FP)** Class 0 wrong as Class 1 5	**True Negative (TN)** Class 0 correct as Class 0 100

Accuracy

90.9%

(TP+TN) / Total

Precision

90.9%

TP / (TP + FP)

Recall

83.3%

TP / (TP + FN)

F1 Score

87.0%

2 * (P * R) / (P + R)

3. Concrete Example: The Exam Scenario

Let's calculate these values using our exam analogy:

- **The Exam has 10 questions.** (Actual Positives = 10)
- **You studied 15 topics.** (Predicted Positives = 15)
- **8 of the topics you studied appeared on the exam.** (True Positives = 8)

Precision (Efficiency)

"How much of my studying was useful?"

$$\text{Precision} = \frac{\text{Useful Topics Studied}}{\text{Total Topics Studied}}$$

$$\text{Precision} = \frac{8}{15} = \mathbf{53.3\%}$$

You wasted time on 7 topics that weren't on the test.

Recall (Coverage)

"Did I cover everything?"

$$\text{Recall} = \frac{\text{Useful Topics Studied}}{\text{Total Questions on Exam}}$$

$$\text{Recall} = \frac{8}{10} = \mathbf{80.0\%}$$

You missed 2 questions because you didn't study them.

4. Python Implementation

Using `classification_report` gives you all these metrics at once.

```
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score

# Suppose y_true and y_pred are your lists of labels
y_true = [0, 1, 0, 1, 0, 0, 1]
y_pred = [0, 1, 0, 0, 0, 1, 1]

# 1. Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix:\n", cm)

# 2. Detailed Report
print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=["Class 0",
"Class 1"]))
```

```
# Output will look like:
#               precision    recall  f1-score   support
#      Class 0       0.75      0.75      0.75         4
#      Class 1       0.67      0.67      0.67         3
```